

Kali: Agentic Cyber-Defense (Setup & User Guide)

Girish Mundra
2024A7PS0993P
BITS Pilani, Pilani Campus

April 30, 2026

Contents

1	Project Overview	2
2	Server Access	2
3	Docker Basics	2
4	Kali Tool Installation	3
5	Python Environment Setup	4
6	Running the Backend	4
7	OWASP ZAP Setup	4
8	Ollama Setup	5
9	Gemini Setup	5
10	API Usage	5
11	System Workflow	5
12	Common Commands Reference	6
13	Troubleshooting	6
14	Project Summary	6

1 Project Overview

This document is a step-by-step guide to reproduce and run the **Kali Agentic Cyber-Defense System**.

The system allows natural language security queries to be executed as real penetration testing workflows inside Dockerized environments using:

- Kali Linux container (nmap, gobuster)
- OWASP ZAP container
- FastAPI backend
- Gemini LLM (planning + analysis)

2 Server Access

Connect to the remote server:

```
ssh username@server_ip
```

3 Docker Basics

3.1 List Images and Containers

```
docker images      # list images
docker ps -a       # list all containers
```

3.2 Load Docker Image

```
docker load -i kali-image.tar
```

3.3 Run Kali Container

```
docker run -it --gpus all --name kali-container kali-image /bin/bash

(or)

docker run -it --gpus all --name kali-container kalilinux/kali-rolling
/bin/bash
```

3.4 Start Existing Container

```
docker start kali-container
```

3.5 Enter Existing Container

```
docker exec -it kali-container /bin/bash
```

3.6 Backup Container/Image

3.6.1 Save Image to File

```
docker save -o kali-backup.tar kali-image
```

3.6.2 Restore Image

```
docker load -i kali-backup.tar
docker run -it --gpus all kali-image /bin/bash
```

4 Kali Tool Installation

4.1 Pull Required Images

Before starting the system, pull required Docker images:

```
docker pull kalilinux/kali-rolling
docker pull ghcr.io/zaproxy/zaproxy:stable
```

4.2 Create Kali Container

Run Kali Linux container with GPU support:

```
docker run -it --gpus all --name kali-container kalilinux/kali-rolling
/bin/bash
```

4.3 Update Kali System

Inside the container, update package lists:

```
apt update
```

4.4 Install Security Tools

Install required penetration testing tools inside Kali container:

```
apt install -y nmap gobuster python3 python3-pip curl git
```

```
apt install -y dirb
```

4.5 Verify Installation

Check installed tools:

```
nmap --version
gobuster -h
python3 --version
```

4.6 Wordlist Location (Gobuster)

Common wordlists used:

```
/usr/share/dirb/wordlists/common.txt  
/usr/share/dirb/wordlists/small.txt
```

4.7 Exit Container

```
exit
```

5 Python Environment Setup

Install python:

```
apt-get update  
  
apt-get install -y python3 python3-pip
```

Activate virtual environment:

```
source venv/bin/activate
```

Install dependencies (if needed):

```
pip install -r requirements.txt
```

6 Running the Backend

Start FastAPI server:

```
GEMINI_API_KEY="your_api_key_here" uvicorn app.main:app --host 0.0.0.0  
--port <port>
```

Stop server:

```
CTRL + C
```

If port is occupied:

```
lsof -i :<port>  
kill -9 <PID>
```

7 OWASP ZAP Setup

Run ZAP container:

```
docker run -u zap -p 8080:8080 ghcr.io/zaproxy/zaproxy:stable zap.sh -  
daemon -host 0.0.0.0 -port 8080
```

Check if running:

```
docker ps
```

Restart ZAP container:

```
docker restart zap-container
```

8 Ollama Setup

Pull Ollama image:

```
docker run -d --gpus all -p 11434:11434 --name ollama-container  
ollama/ollama
```

Check if running:

```
docker ps
```

Pull model inside container:

```
docker exec -it ollama-container ollama pull llama3
```

Test:

```
docker exec -it ollama-container ollama run llama3 "say hello"
```

Verify:

```
curl http://localhost:11434/api/tags
```

9 Gemini Setup

Install Gemini Library in your venv:

```
pip install google-generativeai
```

Check if running:

```
docker ps
```

Restart ZAP container:

```
docker restart zap-container
```

10 API Usage

Send request to backend:

```
curl -X POST http://localhost:<port>/scan \  
-H "Content-Type: application/json" \  
-d '{"input":"scan directory vulnerability scanme.nmap.org"}'
```

11 System Workflow

1. User sends natural language input
2. Gemini LLM converts it into tool execution plan
3. Plan is validated
4. Tools executed inside Docker containers
5. Results collected (nmap, gobuster, zap)
6. Gemini generates final security summary

12 Common Commands Reference

12.1 Docker Navigation

```
docker exec -it kali-container /bin/bash
pwd
ls
```

12.2 Kali Tools

```
nmap -F scanme.nmap.org

gobuster dir -u http://scanme.nmap.org \
-w /usr/share/dirb/wordlists/common.txt -t 10
```

13 Troubleshooting

13.1 Port Already in Use

```
lsof -i :5050
kill -9 <PID>
```

13.2 Docker Container Not Found

```
docker ps -a
docker start kali-container
```

13.3 ZAP Not Responding

```
docker logs zap-container
docker restart zap-container
```

14 Project Summary

This system converts natural language cybersecurity requests into real-world penetration testing operations using:

- LLM-based task planning (Gemini)
- Docker-isolated execution environment
- Automated vulnerability scanning (Nmap, Gobuster, ZAP)
- AI-based result summarization

It serves as a reproducible framework for building autonomous security agents.